LPFM Station Infrastructure Implementation Plan

Raspberry Pi 3A+ Audio Pipeline and Transmitter Control

1. Purpose

This project builds the infrastructure for a Low-Power FM (LPFM) radio station running on a Raspberry Pi 3A+. The station acquires a live audio stream over wifi, routes it to a USB audio dongle connected to an FM transmitter, and controls the transmitter's power via a wifi-connected relay. The system is designed to be reliable, unattended, and self-healing — automatically recovering from stream dropouts or process failures without manual intervention.

The implementation is intentionally generalizable. While designed for a specific station setup, the architecture and configuration patterns should be reusable by anyone building a similar low-power broadcast system.

Logging is handled entirely through `systemd` and `journalctl`, which are standard on Debian-based systems and require no additional infrastructure.

2. Goals and Scope

2.1. Goals

1. Acquire a live audio stream reliably over wifi, with buffering to smooth over brief network hiccups.
2. Route audio to the USB dongle configured as the system audio output device feeding the FM transmitter.
3. Control the transmitter via a wifi power relay — switching it on and off cleanly under software control.
4. Apply scheduling and heuristics to determine when the transmitter should be active (time-of-day rules, stream health as a gate).
5. Interlock transmitter power with stream health — never activate the transmitter without confirmed good audio.
6. Monitor the stream and recover from failure — detect dropouts, restart the stream fetcher, and fall back to local audio if needed.
7. Manage all processes via systemd — auto-start on boot, automatic restarts on crash, journalctl logging with rotation.

8. Drive everything from a single config file — stream URL, schedule, relay address, fallback settings, and tuning parameters all in one place.
9. Allow remote access via SSH for monitoring, control, and troubleshooting.

2.2. Why These Goals Matter

An unattended broadcast system has to be self-sustaining. Without stream monitoring and automatic recovery, a brief network dropout leaves the transmitter radiating silence. Without the interlock, a crashed stream process leaves the transmitter on but carrying nothing. Without systemd management, a power cycle requires manual intervention to restart.

The single config file and remote SSH access ensure that operational changes (adjusting schedule, swapping stream URL, tuning buffer sizes) can be made without touching the code.

2.3. Scope

This implementation covers:

- Stream acquisition: fetching and buffering a remote audio stream.
- Audio routing: configuring the USB audio dongle as the system output and piping stream audio to it.
- Relay control: sending on/off commands to a wifi power relay over the local network.
- Scheduler: time-based rules determining active broadcast windows.
- Heuristics engine: stream health check as a gate on transmitter activation.
- Watchdog: detecting stream failure and triggering recovery or fallback.
- Fallback: playing local audio files when the stream is unavailable.
- Systemd services: unit files for each component, with dependency ordering and restart policies.
- Configuration: a single config file for all tunable parameters.
- Remote access: SSH enabled, no additional tooling required.

2.4. Out of Scope

- Audio level normalization or dynamic range compression.
- Silence detection or dead-air protection.
- Transmitter hardware warm-up/cool-down timing.
- A web UI or API (SSH is sufficient for remote control).
- FCC compliance verification (the operator's responsibility).

3. Development Standards

3.1. Language

Python 3.x throughout. Chosen for its strong Raspberry Pi ecosystem, readable syntax, and ease of writing platform stubs for macOS development.

3.2. Code Style

- Readable over clever — code should be immediately understandable; prioritize clarity over concision.
- Well-commented — explain the *why*, not the what. Comments address intent, constraints, and non-obvious decisions.
- OOP throughout — classes for all meaningful components (stream fetcher, relay controller, scheduler, watchdog, etc.), as reasonable.
- Parallel identifiers — class names, method names, config keys, and log messages all use the same conceptual vocabulary developed for the project (e.g., if the concept is "broadcast window," that term appears consistently everywhere — not "time slot" in one place and "active period" in another).
- No hardcoded values — every tunable parameter lives in the config file. Code contains no magic numbers, URLs, paths, or timing values.

3.3. Documentation

Python equivalent of JSDoc is Google-style docstrings, used consistently:

- File headers — every module begins with a module-level docstring: purpose, author, and a brief description of its role in the system.
- Class docstrings — describe what the class represents and its responsibilities.
- Method/function docstrings — describe purpose, args (with types), return value, and any raised exceptions.

Example pattern:

```Python
    def connect(self, url: str, retries: int = 3) -> bool:
        """Establish a connection to the audio stream.
        Args:
            url: The stream URL to connect to.
```

```
            retries: Number of reconnection attempts before
signaling failure.
        Returns:
            True if connection succeeded, False otherwise.
        Raises:
            StreamConfigError: If the URL is malformed or
unsupported.
        """
```

3.4. Architecture Principles

- Separation of concerns — each module has one job. No module reaches into another's internals.
- Dependency injection over globals — components receive their config and dependencies at construction time.
- Platform abstraction — all Pi-specific calls (audio device, relay control, systemd notify) go through an interface layer so macOS stubs can be swapped in transparently during development and testing.

3.5. Directory Structure

```
lpfm/
├── config/
│   └── config.toml          # all runtime parameters; no defaults in
code
├── lpfm/
│   ├── __init__.py
│   ├── stream_fetcher.py    # stream acquisition and buffering
│   ├── audio_router.py      # audio device configuration and routing
│   ├── relay_controller.py  # wifi relay on/off commands
│   ├── scheduler.py         # broadcast window logic
│   ├── watchdog.py          # stream health monitoring and recovery
│   ├── fallback_player.py   # local audio fallback
│   └── config_loader.py     # config parsing and validation
├── platform/
│   ├── __init__.py
│   ├── raspi.py             # real Pi hardware interfaces
```

```
|   └── macos_stub.py        # development stubs for macOS
├── tests/
|   └── ...
├── systemd/
|   └── *.service            # unit files for deployment
├── requirements.txt
└── main.py                  # entry point; wires components together
```

3.6. Configuration

A single `config/config.toml` file holds all parameters. No defaults are buried in code — if a value isn't in config, the system raises a clear error at startup. Config is loaded once and passed to all components at construction time.

3.7. Dependency Management

Dependencies tracked in `requirements.txt`. A `requirements-dev.txt` covers development-only packages (testing, linting). Target environment is Python 3.9+ to match current Raspberry Pi OS availability.

4. Hardware and Environment Baseline

4.1. Hardware

Component	Role
Raspberry Pi 3A+	Main controller and audio router
USB audio dongle	Analog audio output to FM transmitter input
WiFi power relay	Remotely switches transmitter power via HTTP or MQTT
FM transmitter	Accepts analog audio input; powered by relay
Network (wifi)	Carries the incoming stream and relay control traffic

4.2. Software Environment

- Target OS: Raspberry Pi OS (Debian-based), current stable release.
- Init system: systemd — all services managed as units.

- Logging: journalctl with automatic log rotation via systemd's journal size limits.
- Audio subsystem: ALSA or PulseAudio; USB dongle configured as default output device.
- Development environment: macOS, with stubs standing in for Pi-specific hardware controls (relay, audio device) and faux systemd wrappers for local testing.

4.3. Starting Point

The Pi boots a fresh OS install. No station software is present. The USB dongle and wifi relay are physically connected. The stream source URL is known. The relay's local IP and control interface are known.

5. System Architecture

5.1. Components

stream-fetcher Connects to the remote stream URL, buffers incoming audio, and pipes it to the audio output. Restarts automatically on disconnect. On repeated failure, signals the watchdog to activate fallback mode.

audio-router Configures the USB dongle as the active ALSA/PulseAudio output device. In normal operation this is static configuration, not a running process.

relay-controller Issues HTTP (or MQTT) commands to the wifi power relay to switch the transmitter on or off. Called by the scheduler and the interlock logic. Stateless — each call is a single command.

scheduler Runs on a timer. Consults the config file for active broadcast windows (time-of-day rules). At the start of a window, checks stream health before activating the transmitter. At the end of a window, deactivates the transmitter regardless of stream state.

stream-watchdog Monitors the stream-fetcher process and audio output for signs of failure (process death, stalled bytes, repeated reconnection attempts). On confirmed failure, triggers recovery (restart stream-fetcher) or, after N retries, switches to fallback audio.

fallback-player Plays a local audio file or playlist through the USB dongle when the stream is unavailable. Deactivates and yields to stream-fetcher when the stream recovers.

config A single file (e.g., `config.toml`) holding all tunable parameters: stream URL, broadcast schedule, relay address and credentials, buffer size, retry limits, fallback audio path, and log verbosity.

5.2. Audio Pipeline Flow

```
[REMOTE STREAM SOURCE]
        | (network / wifi)
        v
[stream-fetcher]  ←── restarts on failure (systemd)
        | (piped audio)
        v
[USB audio dongle]  ←── configured as default ALSA output
        | (analog audio signal)
        v
[FM transmitter input]
        | (RF broadcast — only when relay is ON)
        v
[ON AIR]
```

5.3. Transmitter Control Flow

```
[scheduler]  (runs on timer, reads config)
        |── Is current time within a broadcast window?
        |── YES:
        |   [stream-watchdog: health check]
        |       |── Stream healthy?
        |       |── YES → [relay-controller: ON]  →  transmitter powers on
        |       └── NO  → hold off; retry after interval
        └── NO:
            [relay-controller: OFF]  →  transmitter powers off
```

5.4. Failure and Recovery Flow

```
[stream-fetcher]
        |
        |── Dropout or crash detected
        |
[stream-watchdog]
        |
        |── Attempt restart (up to N retries)
        |       |── Recovery SUCCESS → resume normal pipeline
        |       └── Still failing after N retries:
        |               |
        |           [relay-controller: OFF]  ←── interlock: transmitter off
        |               |
        |           [fallback-player: ON]  ←── local audio begins
        |               └── stream-watchdog continues polling
        |                       └── Stream recovers:
```

```
        |                                |
        |                           [fallback-player: OFF]
        |                           [stream-fetcher: restart]
        |                           [relay-controller: ON if in window]
        v
[journalctl logs all transitions]
```

6. Strategic Options and Staged Approach

6.1. Approach: Modular Services

Each component is its own systemd service with defined dependencies. systemd handles ordering, restart policies, and logging for each independently.

- Components restart independently — a stream-fetcher crash doesn't affect the relay controller.
- systemd dependency ordering ensures correct startup sequence.
- Each service logs independently to journalctl; easy to isolate failures.
- Easy to stop, test, or replace one service without touching the others.
- Inter-process coordination via a small shared status mechanism (e.g., a state file or lightweight IPC).

6.2. Staged Implementation

The modular approach is built incrementally. Each phase is independently testable before moving to the next.

Phase 1: Audio Pipeline

- Configure USB dongle as default ALSA output.
- Install and configure stream-fetcher; verify audio plays through dongle.
- No transmitter involvement yet — test purely with headphones or a line-level meter.

Phase 2: Systemd Integration

- Write systemd unit file for stream-fetcher with auto-restart on failure.
- Verify journalctl logging works and log rotation is configured.
- Test that stream-fetcher survives a Pi reboot and an intentional kill.

Phase 3: Relay Control

- Implement relay-controller script; test on/off commands to the wifi relay independently.
- Confirm transmitter powers on and off cleanly.

- Wire relay-controller into manual invocation (SSH command) before automating it.

Phase 4: Scheduler and Heuristics

- Implement scheduler with time-of-day rules from config file.
- Add stream-health check as a gate before relay-controller ON.
- Test schedule transitions: transmitter activates at window start, deactivates at window end.

Phase 5: Watchdog and Fallback

- Implement stream-watchdog to monitor stream-fetcher health.
- Implement fallback-player for local audio on stream failure.
- Wire interlock: transmitter goes off when fallback activates.
- Test full failure-and-recovery cycle end-to-end.

Phase 6: Hardening and Config

- Consolidate all parameters into single config file.
- Review systemd unit dependencies and restart policies.
- Confirm SSH access and remote log tailing work as expected.
- Document the setup for reproducibility.

7. Implementation: Stream Fetcher

7.1. Overview

The stream fetcher connects to the remote `Icecast` audio stream and routes it to the USB audio dongle via `ALSA`. It runs as a managed subprocess wrapping `ffmpeg`, which handles the HTTP stream connection, audio decoding, and `ALSA` output. Brief network dropouts are handled at the `ffmpeg` level via built-in reconnect flags; sustained failures are the responsibility of the watchdog.

7.2. External Dependency

`ffmpeg` must be installed as a system package. It is not a Python dependency:

- Raspberry Pi: `sudo apt install ffmpeg`
- macOS (development): `brew install ffmpeg`

7.3. ffmpeg Command

```
ffmpeg -reconnect  -reconnect_streamed  -reconnect_delay_max  \
       -i <stream_url> \
       -vn -acodec pcm_s16le -ar  -ac  \
       -f alsa <audio_device>
```

- `-reconnect` flags — automatic recovery from brief dropouts at the transport level
- `-vn` — discard any video stream
- `pcm_s16le` — decode to raw 16-bit PCM for ALSA compatibility
- `-f alsa` — output directly to the named ALSA device

7.4. Config

Adds a new `[audio]` section to `config.toml`:

Python
```
[audio]
device = "hw:,"
```

7.5. StreamFetcher Class

Method	Description
`start()`	Launches the ffmpeg subprocess
`stop()`	Terminates ffmpeg cleanly
`is_running()`	Returns True if the subprocess is alive
`restart()`	Calls stop() then start()

7.6. Platform Audio Formats

The ffmpeg output format is platform-specific and must be set in config.toml or abstracted to `.env`.

- Raspberry Pi: `format = "alsa"`, `device = "hw:1,0"`
- macOS (dev): `format = "audiotoolbox"`, `device = "default"`

8. Implementation: Relay Controller

8.1. Overview

The relay controller sends on/off commands to the Shelly 1 Mini Gen4 wifi relay that controls transmitter power. It is stateless — each call is a single HTTP GET request with no persistent connection. The relay controller is called by the scheduler (to activate or deactivate the transmitter at window boundaries) and by the watchdog (to cut power when the stream fails).

8.2. Hardware

The `Shelly 1 Mini Gen4`. Uses the `Gen2/Gen4 JSON-RPC API` over local HTTP. No cloud dependency — all commands go directly to the device on the local network.

8.3. Shelly RPC API

Action	Endpoint
Turn on	`GET /rpc/Switch.Set?id=0&on=true`
Turn off	`GET /rpc/Switch.Set?id=0&on=false`
Query state	`GET /rpc/Switch.GetStatus?id=0`

Authentication is disabled by default on the local network. Digest auth (SHA256) can be enabled on the device if needed — out of scope for now.

8.4. Config

Relay connection details live in .env:

```python
LPFM_RELAY_URL=http://<shelly-ip>
LPFM_RELAY_ON_PATH=/rpc/Switch.Set?id=0&on=true
LPFM_RELAY_OFF_PATH=/rpc/Switch.Set?id=0&on=false
```

8.5. RelayController Class

Method	Description
`turn_on()`	Sends the on command; raises `RelayError` on failure

`turn_off()`	Sends the off command; raises `RelayError` on failure
`get_state()`	Returns `True` if relay is currently on, `False` if off

8.6. Dependency

Requires requests added to requirements.txt.

9. Implementation: Watchdog

9.1. Overview

The watchdog monitors the stream fetcher and restarts it when it fails. If restarts repeatedly fail, it declares the stream dead, cuts the transmitter via the relay controller, and activates the fallback player. It continues retrying in the background and restores normal operation when the stream recovers.

The watchdog runs as a background thread, polling on a fixed interval. It does not inspect audio content — a running `ffmpeg` process is considered a healthy stream.

9.2. Failure and Recovery Sequence

1. Watchdog polls every `N` seconds; detects `ffmpeg` process has died.
2. Immediately attempts to restart the stream fetcher.
3. If still failing after `M` restart attempts (with a cooldown between each), declares stream failure.
4. On stream failure: calls `relay_controller.turn_off()`, then `fallback_player.start()`.
5. Continues polling in the background.
6. When `ffmpeg` stays alive for one full poll cycle: declares stream recovered.
7. On recovery: calls `fallback_player.stop()`, restarts stream fetcher, calls `relay_controller.turn_on()` if currently inside a broadcast window.

9.3. Config

Added to `config.toml` under a new `[watchdog]` section:

```python
[watchdog]
poll_interval_seconds = 10 # how often to check stream health
restart_attempts = 5 # consecutive failures before declaring stream dead
restart_cooldown_seconds = 30 # wait between restart attempts
```

9.4. Watchdog Class

Method	Description
`start()`	Begins polling in a background thread
`stop()`	Stops the polling thread cleanly
`is_stream_healthy()`	Returns True if the stream fetcher is currently running

9.5. Dependencies

Requires `StreamFetcher`, `RelayController`, and `FallbackPlayer` — all injected at construction time. Also requires a reference to the current `Scheduler` to determine whether to restore the transmitter on recovery.

10. Implementation: Fallback Player

10.1. Overview

The fallback player provides audio output when the remote stream is unavailable. It plays audio files from a local directory in shuffled order, looping indefinitely. Like the stream fetcher, it wraps `ffmpeg` as a subprocess — one file at a time. When a file finishes, the next shuffled file begins automatically. When the list is exhausted it reshuffles and repeats.

The fallback player is activated by the `watchdog` on stream failure and deactivated on stream recovery.

10.2. Playback Behavior

1. On `start()`, scan the fallback directory for supported audio files.
2. Shuffle the file list.

3. Play each file in sequence via ffmpeg subprocess.
4. When a file finishes, advance to the next.
5. When the list is exhausted, reshuffle and start again.
6. On `stop()`, terminate the current `ffmpeg` subprocess and exit the playback thread.

10.3. Config

`audio_path` in the `[fallback]` section points to the fallback audio directory:

```python
[fallback]
audio_dir = "content/fallback"
file_extensions = ["mp3", "wav", "flac", "ogg", "aiff"]
```

10.4. FallbackPlayer Class

Method	Description
`start()`	Scans directory, shuffles files, begins playback in a background thread
`stop()`	Terminates current ffmpeg subprocess and stops the playback thread
`is_running()`	Returns True if fallback audio is currently playing

10.5. Notes

- If the fallback directory is empty or missing, start() logs an error and returns without playing rather than raising — silence is preferable to a crash during an already-degraded state.
- Supported file extensions are configurable so the station operator can add formats as needed.

11. Implementation: Scheduler

11.1. Overview

The scheduler makes a daily decision about whether and when to broadcast, driven by a probabilistic risk model with exponential memory. At a configured decision time each day it calculates the current accumulated risk, rolls the dice, and — if broadcasting — picks a random

start and stop time within configured leeway windows. The decision and state are persisted to disk so risk memory survives restarts.

11.2. Daily Cycle

1. At `decision_time` (e.g. 09:00), the scheduler wakes and runs the daily decision.
2. It calculates `accumulated_risk` from the decayed history of past broadcasts.
3. It computes `broadcast_probability = f(accumulated_risk)` — optionally hard-capped to 0 if accumulated risk exceeds a threshold.
4. It rolls a random number; if below `broadcast_probability`, tonight is a broadcast night.
5. If broadcasting: randomly picks `start_time` within `[window_start, window_start + start_leeway_max]` and `stop_time` within `[window_end - stop_leeway_max, window_end]`.
6. Persists the decision to the state file.
7. Optionally sends an email notification with tonight's schedule.
8. At `start_time`, activates the relay (if stream is healthy).
9. At `stop_time`, deactivates the relay.

11.3. Risk Model

Each broadcast generates a risk score:

```
risk = (w_start × start_risk) + (w_stop × stop_risk) + (w_duration × duration_risk) + (w_day × day_risk)
```

Where:

- `start_risk` — how early the start is within the leeway window (earlier = riskier, 0–1)
- `stop_risk` — how late the stop is within the leeway window (later = riskier, 0–1)
- `duration_risk` — broadcast duration as a fraction of the maximum possible duration (0–1)
- `day_risk` — a configurable per-day-of-week multiplier
- `w_*` — configurable weights (should sum to 1.0)

Accumulated risk decays exponentially across days:

```
accumulated_risk = last_broadcast_risk + (decay_factor × previous_accumulated_risk)
```

A `decay_factor` of 0.7 means yesterday's risk still contributes 70%, two days ago 49%, and so on.

11.4. Config

New `[scheduler]` and `[risk]` sections in `config.toml`:

```python
[scheduler]
decision_time = "09:00"
window_start = "18:00"
window_end = "23:00"
start_leeway_max_minutes = 120
stop_leeway_max_minutes = 120

[risk]
decay_factor = 0.7
broadcast_threshold = 0.85 # accumulated risk above this = no
broadcast (0 to disable)
weight_start = 0.30
weight_stop = 0.30
weight_duration = 0.25
weight_day = 0.15
day_weights = { monday=0.5, tuesday=0.5, wednesday=0.5,
thursday=0.6, friday=0.8, saturday=0.9, sunday=0.7 }
```

11.5. State File

Persisted to `state/scheduler.json`:

```json
{
  "accumulated_risk": 0.42,
  "last_updated": "2026-05-06",
  "last_broadcast": {
    "date": "2026-05-05",
    "start": "20:15",
    "stop": "22:45",
    "risk_score": 0.61
  },
  "today": {
```

```
    "decided": true,
    "broadcasting": true,
    "start": "20:30",
    "stop": "22:50"
  }
}
```

11.6. Notifications

At decision time, if broadcasting tonight, the scheduler optionally sends an email with the planned start and stop times. SMTP credentials live in `.env`. Notification can be disabled in config.

11.7. Scheduler Class

Method	Description
`start()`	Begins the background scheduling thread
`stop()`	Stops the thread cleanly
`is_in_broadcast_window()`	Returns True if current time is between today's decided start and stop

12. Implementation: Control Panel

The control panel is a Flask web dashboard running on port 8080, accessible at `http://lpfm.local:8080` on the local network. It runs as a daemon thread inside the main process — no separate service required.

12.1. Features

- **Emergency shutoff** — big red button that immediately cuts transmission and holds until manually cleared; turns green to restore
- **Tonight's broadcast** — editable toggle (Yes/No), start time, and stop time; saved directly to `state/scheduler.json` and wakes the scheduler thread immediately
- **Stream URL override** — set a one-time stream URL that takes effect immediately (switches the live ffmpeg process) and resets automatically after the broadcast ends; Reset button restores the default

- **Accumulated risk** — displays current risk value and broadcast probability with a visual bar
- **Recent history** — table of the last N decisions showing date, start/stop times, risk score, accumulated risk, and on-air status

12.2. Architecture

- `ControlPanel` receives references to `Scheduler` (for `wake()`) and `StreamFetcher` (for `set_url()` / `reset_url()`)
- State reads and writes go directly to `state/scheduler.json`; history reads from `state/history.jsonl`
- The control panel never appends to history — that's the Scheduler's responsibility
- URL validation is handled client-side (HTTP/HTTPS only; warns on playlist extensions)

12.3. Access

Local network only — no authentication. Intended for operator use on the same network as the station.

13. Appendices

13.1. Appendix A: Open Decisions

13.2. Appendix B: Document Style Guide

This section details the standardized formatting applied to the document.

13.2.1. Headers and Section Titles

This style dictates the formatting for all section titles and headings (Heading 1, Heading 2, Heading 3, etc.) to ensure clear hierarchy and visual distinction from body text. All headers must be formatted exclusively using these standard Google Docs heading styles rather than custom text styles.

- **Font Family:** Arial
- **Font Size:** Varied (e.g., 20pt for Heading 1, 16pt for Heading 2, 14pt for Heading 3)
- **Font Weight:** Normal
- **Application:** Applied exclusively to section titles and subtitles, using the designated Docs Heading styles.

Structural Patterns: H1 headings (e.g., "1. Goals and Scope") must use the "Heading 1" doc style; H2 headings (e.g., "1.1. Goals") must use the "Heading 2" doc style; and this hierarchical pattern must be followed for all subsequent heading levels.

13.2.2. Body Paragraphs and Standard Prose

This section defines the baseline formatting for all narrative and descriptive text in the document. Standardized prose ensures optimal readability and consistency across all sections, distinguishing it clearly from headings and technical lists.

- **Font Family:** Arial
- **Font Size:** 11pt
- **Line Spacing:** 1.15
- **Spacing (Before Paragraph):** 0pt
- **Spacing (After Paragraph):** 12pt
- **Application:** Applied exclusively to normal body text, excluding headings, titles, and list items.

13.2.3. Inline Technical References

This style is used for all short code references embedded within the main text, such as variable names, file paths, API routes, and terminal commands. It serves to visually isolate code elements from surrounding narrative text.

- **Font Family:** Roboto Mono (Monospaced)
- **Text Color:** Gemini Green (#188038)
- **Background:** None
- **Use Cases:** Inline code snippets (e.g., `clipUsage` table), variable names (e.g., `audioID`), file paths (e.g., `config/config.js`), and console output text.

13.2.4. Code Block Syntax Highlighting

Background: Light gray (`#f7f7f7` or similar off-white) **Default text:** Dark gray (identifiers, punctuation, table/column names)

Token colors:

- **Keywords** (`ALTER`, `CREATE`, `ADD`, `NOT`, `NULL`, `ON`, `DELETE`): Medium blue (`#0070c1` or similar)
- **Types** (`INT`, `VARCHAR`, `DATETIME`): Bright blue (`#0099cc` or similar)
- **Numeric literals** (`255`, `32`, `191`): Orange-red (`#e06c00` or similar)
- **Comments** (`-- ...`): Muted pink/mauve (`#c678a0` or similar)
- **Language label** (`SQL`): Small-caps, light gray, non-highlighted

Typography: Monospace, medium weight, line height of 1.15. Token colors are distinct but not garish — optimized for readability over visual drama. Before and after line spacing at 0.

```sql
-- Existing 'users' table update (only adding utility columns)
ALTER TABLE users
  ADD COLUMN displayName VARCHAR(255),
  ADD COLUMN lastLoginAt DATETIME;

-- New 'userIdentities' table to support multiple linked OAuth accounts
CREATE TABLE userIdentities (
  userId INT NOT NULL, -- Foreign key to the users table
  oauthProvider VARCHAR(32) NOT NULL,
  oauthId VARCHAR(191) NOT NULL,
  PRIMARY KEY (oauthProvider, oauthId),
  FOREIGN KEY (userId) REFERENCES users(userID) ON DELETE CASCADE
);

-- Allow null names since OAuth providers may not supply them
ALTER TABLE users
```